

BoJio

BoJio (*student proposal*)

Assessment: ✓ Fit · **Category:** Social Media · **Assessed:** 2026-09-22 ·
Status: awaiting instructor approval

Proposal (as submitted): *BoJio* is a **mobile social discovery + group-planning app** that turns interesting activities into fair, practical outings. Users discover activities via a **photo-first feed** or real-world venue data, save them, and hit "**Plan This**" to create a group outing; the group provides locations, availability, budget, interests and travel preferences. An **AI-assisted recommendation** component interprets preferences and retrieves activities, real-world place/travel data supply venue + journey info, and a **performant C++ planning engine** evaluates constraints (time, budget, travel distance, group fairness) to generate multiple feasible itineraries; users compare plans, see trade-offs, pick one, and view it as a **map + timeline** with reminders. The architecture separates recommendation, external-data integration and deterministic C++ planning so each subsystem is testable independently. Named load-bearing C++: a **Multi-Objective C++ Itinerary Planner** — receives a structured `PlanningProblem`, removes activities violating **hard constraints** (availability, opening hours, budget, travel feasibility; constraint-satisfaction with **forward-checking** / **AC-3** pruning), then evaluates meeting points + activity sequences against **weighted soft objectives** (group travel fairness, preference match, total travel time, cost, activity diversity) via **heuristic search**, and returns the ranked **Top-K** itineraries. Named technical features: the planner; **offline-aware local caching + sync** (Android Room cache for feeds/saved/recent plans, shown immediately while refreshing); **vector-based semantic retrieval** (preferences + activity descriptions → embeddings in **PostgreSQL + pgvector**, similarity search matching intent — "something chill and creative" → pottery / art jamming / board-game cafés); and **real-world geospatial + travel integration** (Google Places + Routes + geospatial queries normalised into a common internal format). Tool: a **planning Simulator + Config Tool** (synthetic groups/locations/budgets/preferences, edit activity data, configure planner weights/constraints, run the engine, inspect Top-K + score breakdowns; edge-cases like conflicting schedules / impossible budgets /

unfair travel; planner params + activity data as external config). Design features: photo-first social discovery; the "Plan This" group flow (must-do vs optional); alternative-plan comparison (Balanced / Fairest Travel / Lowest Cost / Best Preference Match, vote + select); and a visual itinerary + explanation (timeline + map + warnings + group-fit indicators). Team: **6 CS + 1 Design + 0 Art**.

Note (category): BoJio's front is **social** — a photo-first community feed, friends, the "Plan This" group flow and collaborative voting — so it is assessed as **Social Media**; the C++ multi-objective planner, semantic retrieval and external-data integration are the engine behind the social planning. (A Visualization & Analytics reading was considered for the planner + comparison/trade-off views.)

Note (module completeness): the design already leans on a backend + **PostgreSQL/pgvector** + Google Places/Routes — this is exactly the module's model (external DB mandatory; a Linux server set up by M2, full use T2; Firebase optional for auth). *Platform:* it's an explicit **Android** app (Room cache) — Native Android, Pixel 9a. *Scope caution:* the ambitions are broad (a real constraint-satisfaction planner + a vector-retrieval pipeline + external-API integration + offline sync); make the **deterministic C++ planner the load-bearing star** and let the AI-retrieval + full offline-sync flex (see scope note).

References:

- [Meetup](#) (the benchmark for discovering and organising **group activities/events** — browse activities by interest/location on a map, save them and coordinate a group — the social-discovery + group-outing precedent)
- [Wanderlog](#) — collaborative trip planner (a **shared itinerary** app where a group co-plans, plots stops on an interactive **map**, and views the plan on a timeline — the collaborative-itinerary + map/timeline precedent)
- [Pinterest](#) (the **photo-first visual discovery** platform — explore and **save** ideas through images rather than text-first search — the photo-first-feed + save precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	BoJio
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the C++ planning service via JNI, OpenGL ES for the map/timeline/comparison visuals in a GLSurfaceView, Kotlin app shell + Room cache; device: Pixel 9a) — a photo-first, on-the-go social app; feed/plan/profile screens can be pure Native UI. The Native PC build is the dev/tools vehicle (the planning Simulator + Config Tool). **Extended:** the team's **Linux server + PostgreSQL/pgvector** (set up by M2, full use T2) hosting the vector retrieval, the normalised place/route data and the planning service; Firebase optionally for auth/accounts.

Front end: A 2D app — the photo-first discovery feed, the "Plan This" group-input flow, the alternative-plan comparison cards, and the visual itinerary (synchronised timeline + map with travel legs, timings, costs, warnings + group-fit indicators). **Languages:** Native — Kotlin Native Android UI + the C++ planner via JNI (Android Studio); OpenGL ES for the custom map/timeline/comparison visuals in a GLSurfaceView; feed/profile screens can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: C++ + services — the deterministic multi-objective planner (hard-constraint pruning via forward-checking/AC-3, weighted soft-objective scoring, heuristic search, Top-K ranking) — plus a Linux server with PostgreSQL/pgvector (semantic vector retrieval), the Google Places/Routes integration normalised into a common format, and the Android Room offline cache + sync. **Languages:** C++ (the planning service — load-bearing); backend + pgvector on the Linux server; Firebase optional.

Backend scope: Client + services now, server-hosted (T2) — the planner + retrieval + external-data seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the **C++ planning service** is the star — it ingests a `PlanningProblem`, prunes hard-constraint-violating candidates (forward-checking / AC-3), scores meeting points + sequences against weighted soft objectives (fairness, preference, travel, cost, diversity) via heuristic search, and returns ranked Top-K itineraries; it is deterministic and the app's main performance-sensitive component.
- *Data-driven:* planner weights + constraints, activity definitions, and the group/preference schema are all external config/data — a new weight or activity is a data change (authored in the simulator/config tool).
- *2D:* the feed, the comparison cards, and the synchronised **map + timeline** with travel legs/timings/costs (custom-rendered) satisfy the 2D gate. *3D:* not needed.
- *Front-end + Back-end:* a genuine split — Android client (+ Room cache) over a C++ planner + Linux/pgvector backend + external APIs; the fetch/sync + external-data flows are **Application Communication** ✓.
- *Custom tool:* the **planning Simulator + Config Tool** — synthetic groups/activities, tunable planner weights/constraints, run + inspect Top-K with score breakdowns, and edge-case testing.
- *Networking:* asynchronous social (share/vote/sync), no real-time multiplayer, so a Networking System is optional (●). *External DB:* central (pgvector +

normalised place data + plans/social). *Auth* » / *Analytics* »: accounts + engagement analytics are natural but module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Social Media pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the **C++ planning service** + core logic.

Tech & dependencies

- **Tech: Web build:** A C++ performant service ingesting a `PlanningProblem`, pruning hard-constraint violations (forward-checking / AC-3) and scaffolding the heuristic search; SDL2/3 host for the dev build; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ planner via JNI); OpenGL ES map/timeline in a `GLSurfaceView`; some screens pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the ranking (M1G9) and external data (M1G8) feed it.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the `PlanningProblem` schema + planner config + offline-cache model.

Tech & dependencies

- **Tech:** A data model with the `PlanningProblem` (members, availability, budget, preferences, travel tolerance, candidates, opening hours, travel times), planner weights/constraints and activity definitions as external JSON config — a new weight or activity is a data change; the shared model the Room cache mirrors.
- **Prerequisites:** G1 (planner & entity hooks).
- **Feeds into:** M2G2 (the simulator/config tool), M1G8 (external-data normalisation), M1G9 (scoring inputs).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui/overlay inspectors (constraint pruning, score breakdowns, Top-K), Tracy profiling of the planner over many candidates/combinations, Catch2 tests (hard-constraint correctness,

deterministic ranking, edge cases — conflicting schedules / impossible budgets), on a GitHub Actions Windows+Linux matrix.

- **Prerequisites:** G1 (planner to instrument).
- **Feeds into:** M2G2 (the simulator surfaces these); M2G3 (CI hardening).

- **G8 Firebase Integration — Identity & Data (+ Geospatial/Travel Integration & Offline Cache)** — external data + the Room cache/sync.

Tech & dependencies

- **Tech:** Accounts + saved activities/plans in the store (Firebase and/or Linux/Postgres); the Google Places + Routes integration normalised into a common internal format; and the Android Room offline cache showing feeds/saved/recent plans immediately while refreshing from the backend.
- **Prerequisites:** G2 (data model).
- **Feeds into:** M1G9 (activity candidates), the planner (M1G1); M2G8 (retrieval matured).

- **G9 Matching / Ranking Engine — Foundation (Semantic Retrieval + Multi-Objective Scoring)** — the AI-retrieval + Top-K ranking.

Tech & dependencies

- **Tech:** Embeddings for preferences + activity descriptions in PostgreSQL/pgvector with semantic similarity search (intent match), plus the multi-objective weighted-scoring layer (fairness, preference, travel, cost, diversity) that ranks the planner's feasible candidates into Top-K.
- **Prerequisites:** G2 (activity/preference model).
- **Feeds into:** M2G8 (Ranking II — candidate generation + evaluation metrics); the comparison cards.

- **G14 Profile & Social Graph System** — friends + groups for "Plan This".

Tech & dependencies

- **Tech:** Firestore/Postgres social-graph modelling (friends, groups), the "select friends" group creation, and shared plans/votes, with privacy in mind.
- **Prerequisites:** Identity & data (M1G8).
- **Feeds into:** The group planning flow; M2G13 (group-fit analytics).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — the planner perf + the Linux/pgvector server seam.

Tech & dependencies

- **Tech: Web build:** Profiling + optimization of the planner (pruning efficiency, threading over candidate batches, memory pools), sanitizers, and on-device validation (Pixel 9a); the Linux + PostgreSQL/pgvector server (retrieval + normalised data + planning) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ planner via JNI); OpenGL ES map/timeline in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a responsive planner.

- **G2 ★ Content Editor — Core (Planning Simulator + Config Tool)** — the named dev/design tool.

Tech & dependencies

- **Tech:** Dear ImGui (or a small desktop UI): create synthetic groups/locations/budgets/preferences, add/edit activity data, configure planner weights + constraints, run the C++ engine, and inspect the Top-K itineraries with score breakdowns; round-trip external config.
- **Prerequisites:** G2 (config/data model), M1G1 (the planner).
- **Feeds into:** Rapid tuning + edge-case testing; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (constraint correctness, deterministic ranking, retrieval, edge cases via the simulator) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ planner via JNI); OpenGL ES map/timeline in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G8 Ranking / Matching Engine II — Planner & Semantic Retrieval Matured** — candidate generation + evaluation.

Tech & dependencies

- **Tech:** Matured candidate generation (meeting points + sequences), refined multi-objective scoring (weighted features, cosine similarity for retrieval), and evaluation metrics + the presets (Balanced / Fairest Travel / Lowest Cost / Best Preference Match) that drive the comparison cards.
- **Prerequisites:** M1G9 (retrieval + scoring foundation).
- **Feeds into:** The alternative-plan comparison; the quality of the Top-K.
- **G11 Search & Discovery Engine** — the photo-first activity discovery.

Tech & dependencies

- **Tech:** Discovery/search over activities (semantic + filters over the normalised place data), an incremental index and a query layer, feeding the photo-first feed and the "save / Plan This" flow (custom C+ +/services).
- **Prerequisites:** M1 content/data model; the retrieval (M1G9).
- **Feeds into:** The discovery feed; the planner's candidate pool.
- **G13 Engagement Analytics Dashboards — Plan Comparison, Trade-offs & Explanations** — the visual itinerary + group-fit viz.

Tech & dependencies

- **Tech:** Custom rendering (no chart libs) for the comparison cards (trade-off metrics per plan), the synchronised map + timeline (travel legs, timings, costs), and the explanations / warnings / group-fit indicators that justify each itinerary.
- **Prerequisites:** M2G8 (scored plans), M1G14 (group data).
- **Feeds into:** The "understand why" showcase; informed group voting.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 0 Art (8) — **as proposed**, and workable: a planning/social **utility** app has a **light-to-moderate** art surface (the photo-first feed uses user/venue photos, so the design work is UI, the comparison cards, the map/timeline styling and iconography), which the **D11 conditional** can carry. That said, a **solo designer + 0 artists** is the case the design pool flags ("must ask the instructor which conditional bends", see [design-goals-pool.pdf](#)), and the **design/UX load here is heavy** — the "Plan This" input flow, the alternative-plan comparison UX (making trade-offs legible), and the explanation/warnings design are the make-or-break of the product — so a **2nd designer is recommended** (one owning the discovery/social + planning-input UX, the other the comparison/explanation UX + the D11 visual surface). *Scope note (advisory):* a real multi-objective constraint-satisfaction planner + a vector-retrieval pipeline + external-API integration + offline sync + a simulator tool is a **large, ambitious CS scope** — the **deterministic C++ planner (hard constraints + weighted scoring + Top-K) + the simulator tool** is the load-bearing star and the M1/M2 floor; make the **AI/semantic retrieval and full offline-sync the stretch** (a keyword/filter retrieval is an acceptable fallback if pgvector embeddings slip). Getting the [PlanningProblem](#) schema + planner architecture right early (as the team's own "separate subsystems" plan intends) is the single biggest de-risker.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the light Art surface is carried by the Design-owned D11 conditional); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the social-discovery + fair-group-planning USP, the "Plan This" flow, the alternative-plan comparison and the visual-itinerary explanation; the problem — friend groups struggle to plan fair outings; primary benchmark e.g. Meetup / Wanderlog, and a Singapore benchmark)
- M1D2 Core-Loop Interactive Prototype (a clickable prototype of discover → Plan This → compare plans → select → view itinerary, proving the UX before the planner is deep) · Figma

- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the visual identity: UI system, the comparison-card + map/timeline visual language, iconography; 2 mockups.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the UI, comparison cards, map/timeline and explanations implemented in-engine, no placeholders.

- **UI/UX M1:**

- M1U1 ★ UI Standards & Wireframe Baseline — standards for the photo-first feed, the "Plan This" multi-input flow, the comparison cards and the itinerary timeline/map, plus a wireframe baseline; making **trade-offs legible** is the priority.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the feed, "Plan This" flow, comparison + voting and the visual itinerary navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated with real friend groups (does the comparison actually help them agree on a fair plan?), polished on the Pixel 9a.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial audio from the week-5 needs list present & working (UI feedback, plan-ready, vote and reminder/"hit the road" notification cues), suitable and correctly triggered — modest for a utility app.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — the app's audio set (UI + plan-ready + vote + reminder/notification cues), suited to a mobile social/utility app.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the playback the build needs (multi-channel; a distinct plan-ready + reminder cue).

- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the planning Simulator + Config Tool), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*